

Le cahier des charges

Plateforme Intelligente de Gestion Universitaire

Réalisé par :

Faty Malo Samb

N06275720202

Fatymalo.samb@unchk.edu.sn

Lien : <https://github.com/fatymalo/smart-university>

1. Présentation du projet

1.1 Contexte

La gestion d'un établissement universitaire implique la manipulation quotidienne d'un grand nombre d'informations concernant les étudiants, les enseignants, les cours, les emplois du temps, les notes, les paiements et les documents administratifs.

L'objectif du projet **Smart University** est de concevoir et développer une plateforme web moderne permettant de centraliser et de simplifier la gestion administrative et académique d'une université.

La plateforme permettra aux administrateurs de gérer les principales ressources universitaires depuis une interface unique, moderne, sécurisée et facile à utiliser.

1.2 Problématique

Les établissements universitaires peuvent rencontrer plusieurs difficultés :

- gestion manuelle ou dispersée des informations ;
- difficulté de suivi des étudiants ;
- gestion complexe des enseignants et des cours ;
- manque de centralisation des notes et paiements ;
- difficulté de communication entre les différents acteurs ;
- absence de suivi précis des opérations administratives ;
- génération manuelle de rapports ;
- manque de visibilité sur les activités du système.

Smart University vise à répondre à ces problèmes grâce à une plateforme centralisée.

2.Objectifs du projet

2.1 Objectif général

Développer une plateforme web intelligente permettant de gérer efficacement les activités administratives et académiques d'une université.

2.2 Objectifs spécifiques

La plateforme devra permettre de :

- gérer les étudiants ;
- gérer les enseignants ;
- gérer les cours ;
- gérer les emplois du temps ;
- gérer les notes ;
- gérer les paiements ;
- gérer la bibliothèque ;
- gérer les messages ;
- gérer les notifications ;
- générer des rapports ;
- assurer la traçabilité des opérations avec un système d'audit ;
- gérer les profils utilisateurs ;
- gérer les paramètres de la plateforme ;
- sécuriser l'accès aux différentes fonctionnalités.

3.Utilisateurs du système

3.1 Administrateur

L'administrateur dispose d'un accès complet à la plateforme.

Il peut :

- gérer les étudiants ;
- gérer les enseignants ;
- gérer les cours ;
- gérer les emplois du temps ;
- gérer les notes ;
- gérer les paiements ;
- gérer la bibliothèque ;
- consulter les messages ;
- consulter les notifications ;
- générer les rapports ;
- consulter les journaux d'audit ;
- gérer son profil ;
- modifier les paramètres.

3.2 Personnel universitaire

Selon les droits attribués, le personnel pourra consulter ou modifier certaines informations :

- étudiants ;
- cours ;
- notes ;
- emplois du temps ;
- messages ;
- documents.

3.3 Étudiant

Dans une évolution future, un espace étudiant pourra permettre de :

- consulter son profil ;
- consulter ses cours ;
- consulter son emploi du temps ;
- consulter ses notes ;
- consulter ses paiements ;
- recevoir des notifications ;
- communiquer avec l'administration.

4.Fonctionnalités principales

4.1 Authentification

La plateforme doit proposer une page de connexion sécurisée.

Fonctionnalités :

- connexion ;
- déconnexion ;
- protection des pages ;
- contrôle d'accès ;
- gestion de session ;
- redirection vers le tableau de bord après connexion.

Les pages protégées doivent être accessibles uniquement aux utilisateurs authentifiés.

5. Tableau de bord

Le Dashboard constitue la page principale de l'administration.

Il doit présenter :

- nombre total d'étudiants ;
- nombre total d'enseignants ;
- nombre de cours ;
- nombre de paiements ;
- statistiques générales ;
- activités récentes ;
- notifications récentes ;
- raccourcis vers les fonctionnalités principales.

Le tableau de bord doit présenter les informations sous forme de cartes statistiques, tableaux et graphiques lorsque nécessaire.

6. Gestion des étudiants

La plateforme doit permettre la gestion complète des étudiants.

Fonctionnalités

- ajouter un étudiant ;
- modifier un étudiant ;
- supprimer un étudiant ;
- consulter les informations d'un étudiant ;
- rechercher un étudiant ;
- filtrer les étudiants ;
- afficher les étudiants sous forme de tableau ;
- gérer la pagination ;
- gérer le statut de l'étudiant.

Informations

Un étudiant peut posséder :

- matricule ;
- nom ;
- prénom ;
- email ;
- téléphone ;
- date de naissance ;
- filière ;
- niveau ;
- adresse ;
- statut ;
- photo.

7.Gestion des enseignants

La plateforme doit permettre de gérer les enseignants.

Fonctionnalités

- ajouter un enseignant ;
- modifier un enseignant ;
- supprimer un enseignant ;
- consulter un enseignant ;
- rechercher un enseignant ;
- filtrer les enseignants ;
- gérer les spécialités ;
- gérer les départements.

Informations

- matricule ;
- nom ;
- prénom ;
- email ;
- téléphone ;
- spécialité ;
- département ;
- statut ;
- photo.

8.Gestion des cours

Le module Cours permet de gérer les enseignements proposés par l'université.

Fonctionnalités

- créer un cours ;
- modifier un cours ;
- supprimer un cours ;
- consulter un cours ;
- rechercher un cours ;
- affecter un enseignant ;
- définir le nombre de crédits ;
- définir le niveau ;
- associer une filière.

Informations

- code du cours ;
- nom ;
- description ;
- enseignant ;
- filière ;
- niveau ;
- crédits ;
- semestre.

9. Gestion des emplois du temps

Le module Emploi du temps permet d'organiser les cours.

Fonctionnalités

- créer un planning ;
- modifier un planning ;
- supprimer un planning ;
- consulter le planning ;
- rechercher un cours ;
- filtrer par jour ;
- filtrer par enseignant ;
- filtrer par salle.

Informations

- cours ;
- enseignant ;
- salle ;
- date ;

- heure de début ;
- heure de fin ;
- groupe ;
- niveau.

10. Gestion des notes

Le module Notes permet de gérer les résultats académiques.

Fonctionnalités

- ajouter une note ;
- modifier une note ;
- supprimer une note ;
- consulter les notes ;
- rechercher un étudiant ;
- calculer les moyennes ;
- afficher les résultats ;
- gérer les semestres.

Informations

- étudiant ;
- cours ;
- note ;
- coefficient ;
- semestre ;
- année universitaire ;
- moyenne.

11. Gestion des paiements

Le module Paiements permet de suivre les frais universitaires.

Fonctionnalités

- enregistrer un paiement ;
- modifier un paiement ;
- consulter un paiement ;
- rechercher un paiement ;
- filtrer les paiements ;
- consulter les paiements en attente ;
- consulter les paiements effectués ;
- générer des rapports financiers.

Informations

- étudiant ;
- référence ;
- montant ;
- date ;
- mode de paiement ;
- statut ;
- description.

12. Gestion de la bibliothèque

Le module Bibliothèque permet de gérer les ressources documentaires.

Fonctionnalités

- ajouter un livre ;
- modifier un livre ;
- supprimer un livre ;
- rechercher un livre ;
- consulter la disponibilité ;
- enregistrer un emprunt ;
- enregistrer un retour ;
- suivre les emprunts.

Informations

- titre ;
- auteur ;
- ISBN ;
- catégorie ;
- année ;
- disponibilité ;
- nombre d'exemplaires.

13. Gestion des messages

Le module Messages permet de faciliter la communication interne.

Fonctionnalités

- consulter les messages ;
- envoyer un message ;

- répondre à un message ;
- supprimer un message ;
- rechercher un message ;
- afficher les messages non lus ;
- afficher le compteur de messages non lus.

La navbar doit afficher le nombre de messages non lus.

Exemple :

 **Messages : 1**

Le compteur doit être mis à jour automatiquement lorsque l'utilisateur consulte ou reçoit un message.

14. Gestion des notifications

Le module Notifications permet d'informer l'utilisateur des événements importants.

Exemples

- nouvel étudiant enregistré ;
- nouveau message ;
- paiement effectué ;
- nouveau cours ;
- modification d'un emploi du temps ;
- nouvelle note ;
- opération administrative.

Fonctionnalités

- consulter les notifications ;
- marquer une notification comme lue ;
- supprimer une notification ;
- afficher les notifications non lues ;
- afficher le nombre de notifications non lues.

La navbar doit afficher le compteur des notifications.

Exemple :

 **Notifications : 3**

15. Gestion des rapports

Le module Rapports permet de générer des documents statistiques et administratifs.

Rapports prévus

- rapport des étudiants ;
- rapport des enseignants ;
- rapport académique ;
- rapport des absences ;
- rapport financier ;
- rapport des inscriptions.

Fonctionnalités

- générer un rapport ;
- consulter un rapport ;
- télécharger un rapport ;
- imprimer un rapport ;
- rechercher un rapport ;
- filtrer les rapports ;
- exporter les données.

Les formats prévus sont notamment :

- PDF ;
- Excel.

16. Système d'audit

Le module Audit permet de conserver une trace des opérations effectuées dans la plateforme.

Informations enregistrées

- utilisateur ;
- action ;
- module ;
- date ;
- heure ;
- adresse IP ;
- description.

Exemples d'actions

- connexion ;

- déconnexion ;
- ajout d'étudiant ;
- modification d'étudiant ;
- suppression d'étudiant ;
- ajout de cours ;
- modification d'une note ;
- enregistrement d'un paiement.

L'audit permettra d'améliorer la sécurité et la traçabilité du système.

17. Gestion du profil

Chaque utilisateur doit pouvoir gérer son profil.

Fonctionnalités

- consulter son profil ;
- modifier ses informations ;
- modifier son numéro de téléphone ;
- modifier son adresse email ;
- modifier sa photo ;
- modifier son mot de passe ;
- consulter les informations de son compte.

18. Paramètres

Le module Paramètres permettra de personnaliser la plateforme.

Fonctionnalités

- gestion du compte ;
- paramètres de sécurité ;
- préférences de notifications ;
- paramètres généraux ;
- gestion des autorisations ;
- configuration de l'application.

19. Interface utilisateur

L'interface doit être :

- moderne ;
- professionnelle ;
- responsive ;
- intuitive ;
- rapide ;
- cohérente sur toutes les pages.

Éléments principaux

La plateforme comportera :

Sidebar

Elle permettra d'accéder aux :

- Dashboard ;
- Étudiants ;
- Enseignants ;
- Cours ;
- Emploi du temps ;
- Notes ;
- Paiements ;
- Bibliothèque ;
- Rapports ;
- Audit ;
- Paramètres.

Navbar

La barre supérieure comportera :

- recherche ;
- notifications ;
- messages ;
- profil utilisateur ;
- déconnexion.

20. Architecture technique

20.1 Frontend

Le frontend sera développé avec :

- Angular ;
- TypeScript ;
- HTML5 ;

- CSS3 ;
- Bootstrap ;
- Bootstrap Icons.

L'application utilise une architecture Angular basée sur des composants standalone.

20.2 Backend

Le backend sera développé avec :

- Java ;
- Spring Boot ;
- Spring Web ;
- Spring Data JPA ;
- Spring Security.

Le backend exposera des API REST permettant au frontend de communiquer avec la base de données.

20.3 Base de données

Une base de données relationnelle sera utilisée.

Elle pourra contenir notamment les tables :

- users ;
- students ;
- teachers ;
- courses ;
- schedules ;
- grades ;
- payments ;
- books ;
- messages ;
- notifications ;
- reports ;
- audit_logs.

21. Sécurité

La plateforme doit assurer :

- authentification des utilisateurs ;
- autorisation selon les rôles ;
- protection des routes ;
- validation des données ;

- sécurisation des mots de passe ;
- gestion des sessions ;
- traçabilité des opérations ;
- protection des données personnelles.

Le frontend utilise actuellement un `authGuard` pour protéger les routes administratives.

22. API REST

Le backend devra exposer des endpoints REST.

Exemples :

```
GET    /api/students
GET    /api/students/{id}
POST   /api/students
PUT    /api/students/{id}
DELETE /api/students/{id}
```

Pour les enseignants :

```
GET    /api/teachers
POST   /api/teachers
PUT    /api/teachers/{id}
DELETE /api/teachers/{id}
```

Pour les messages :

```
GET    /api/messages
POST   /api/messages
PUT    /api/messages/{id}/read
DELETE /api/messages/{id}
```

Pour les notifications :

```
GET    /api/notifications
PUT    /api/notifications/{id}/read
DELETE /api/notifications/{id}
```

23. Performances

La plateforme devra être conçue pour :

- charger rapidement les pages ;
- limiter les requêtes inutiles ;
- utiliser la pagination pour les grandes listes ;
- optimiser les recherches ;
- éviter les doublons de données ;

- permettre l'évolution du nombre d'utilisateurs.

24. Responsive Design

L'application doit fonctionner correctement sur :

- ordinateur ;
- tablette ;
- smartphone.

Les tableaux, formulaires, cartes et menus doivent s'adapter aux différentes tailles d'écran.

25. Tests

Des tests devront être réalisés sur les fonctionnalités principales.

Tests frontend

- navigation ;
- formulaires ;
- recherche ;
- pagination ;
- affichage des données ;
- boutons ;
- messages ;
- notifications ;
- authentification.

Tests backend

- API REST ;
- CRUD ;
- authentification ;
- autorisation ;
- validation ;
- gestion des erreurs.

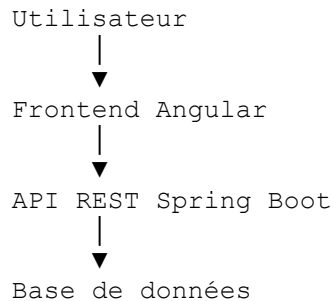
Tests d'intégration

Vérification de la communication :

Angular → API Spring Boot → Base de données

26. Déploiement

La plateforme pourra être déployée selon l'architecture suivante :



Le frontend pourra être hébergé sur un serveur web et le backend sur un serveur applicatif.

27. Critères de réussite

Le projet sera considéré comme fonctionnel lorsque :

- l'utilisateur peut se connecter ;
- les routes protégées sont sécurisées ;
- le Dashboard fonctionne ;
- les étudiants peuvent être gérés ;
- les enseignants peuvent être gérés ;
- les cours peuvent être gérés ;
- les emplois du temps peuvent être gérés ;
- les notes peuvent être gérées ;
- les paiements peuvent être gérés ;
- la bibliothèque fonctionne ;
- les messages fonctionnent ;
- les notifications fonctionnent ;
- les rapports peuvent être générés ;
- les opérations sont enregistrées dans l'audit ;
- le profil peut être modifié ;
- les paramètres sont accessibles ;
- mais le frontend ne communique pas avec mon backend ;
- les données sont persistées dans la base de données.

28. Planning prévisionnel

Phase	Travail
Phase 1	Analyse et cahier des charges
Phase 2	Conception UML et architecture
Phase 3	Création du frontend Angular
Phase 4	Développement des pages
Phase 5	Développement du backend Spring Boot
Phase 6	Création de la base de données
Phase 7	Développement des API REST
Phase 8	Connexion Angular / Spring Boot
Phase 9	Authentification et sécurité
Phase 10	Tests et correction des erreurs
Phase 11	Documentation
Phase 12	Déploiement et présentation

29. Conclusion

Le projet **Smart University** a pour objectif de fournir une solution moderne et centralisée pour la gestion universitaire.

La plateforme permettra de regrouper les principales fonctionnalités administratives et académiques dans une seule application.

L'utilisation d'**Angular** pour le frontend et de **Spring Boot** pour le backend permettra de construire une architecture moderne, évolutive, sécurisée et adaptée aux besoins d'un établissement universitaire.

À terme, Smart University pourra évoluer vers une plateforme complète intégrant différents profils utilisateurs, des notifications en temps réel, une messagerie avancée, des statistiques intelligentes et des fonctionnalités mobiles.